



Programa de Pós-graduação em
INFORMÁTICA



PUC Minas



Teoria dos Grafos e Computabilidade

— Especificação e validação formal de software —

Silvio Jamil F. Guimarães

Graduate Program in Informatics

Image and Multimedia Data Science Laboratory – IMScience

Pontifical Catholic University of Minas Gerais – PUC Minas



Programa de Pós-graduação em
INFORMÁTICA



PUC Minas



Teoria dos Grafos e Computabilidade

— Especificação de sistemas —

Silvio Jamil F. Guimarães

Graduate Program in Informatics

Image and Multimedia Data Science Laboratory – IMScience

Pontifical Catholic University of Minas Gerais – PUC Minas

Exemplo 1

Encontrar a proposição que traduz a seguinte sentença:

Você não pode andar de patins se você tem menos do que 1,20m, a não ser que você tenha mais do que 16 anos

► Definindo:

q: “você pode andar de patins”

r: “você tem menos do que 1,20m”

s: “você tem mais do que 16 anos”

► a sentença pode ser traduzida por:

$$p : (r \wedge \neg s) \rightarrow \neg q$$

Especificação de sistemas

Traduzir sentenças de linguagem natural para linguagem lógica é parte essencial da especificação de sistemas de hardware e software.

Especificação de sistemas

Traduzir sentenças de linguagem natural para linguagem lógica é parte essencial da especificação de sistemas de hardware e software.

Exemplo 2

Expresse a especificação como uma proposição composta

A resposta automática não pode ser enviada quando o sistema de arquivos está cheio

Traduzir sentenças de linguagem natural para linguagem lógica é parte essencial da especificação de sistemas de hardware e software.

Exemplo 2

Expresse a especificação como uma proposição composta

A resposta automática não pode ser enviada quando o sistema de arquivos está cheio

► Definindo:

q: “a resposta automática pode ser enviada”

r: “o sistema de arquivos está cheio”

► a especificação pode ser traduzida por:

$$p : r \rightarrow \neg q$$

Dúvidas?

Especificação e validação formal de
– Especificação de sistemas –



Programa de Pós-graduação em
INFORMÁTICA



PUC Minas



Teoria dos Grafos e Computabilidade

— Verificação de programas —

Silvio Jamil F. Guimarães

Graduate Program in Informatics

Image and Multimedia Data Science Laboratory – IMScience

Pontifical Catholic University of Minas Gerais – PUC Minas

Demonstrações matemáticas de correção para verificar que um dado algoritmo funciona de acordo com as suas especificações

Teste *inserção de dados (no programa de computador) que satisfaçam as condições prévias e verificamos se as saídas satisfazem as condições posteriores.*

Verificação de Programas

Demonstrações matemáticas de correção para verificar que um dado algoritmo funciona de acordo com as suas especificações

Teste *inserção de dados (no programa de computador) que satisfaçam as condições prévias e verificamos se as saídas satisfazem as condições posteriores.*

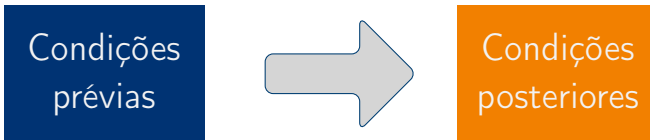
Verificação *prova matemática de que, não importa qual dado de entrada (inserido no programa), a saída sempre irá satisfazer as condições posteriores.*

Verificação de Programas

Demonstrações matemáticas de correção para verificar que um dado algoritmo funciona de acordo com as suas especificações

Teste *inserção de dados (no programa de computador) que satisfaçam as condições prévias e verificamos se as saídas satisfazem as condições posteriores.*

Verificação *prova matemática de que, não importa qual dado de entrada (inserido no programa), a saída sempre irá satisfazer as condições posteriores.*



Verificando Algoritmos Recursivos

A verificação de algoritmos recursivos pode ser fácil a partir da indução.

A verificação de algoritmos recursivos pode ser fácil a partir da indução.

Exemplo 3

Provar que um algoritmo de reverter uma cadeia está correto (1/2)

Condições prévias $s = c_1, c_2, c_3, \dots, c_n$ é uma cadeia ($n \geq 0$)

Condições posteriores $s = c_n, c_{n-1}, c_{n-2}, \dots, c_1$

Verificando Algoritmos Recursivos

A verificação de algoritmos recursivos pode ser fácil a partir da indução.

Exemplo 3

Provar que um algoritmo de reverter uma cadeia está correto (1/2)

Condições prévias $s = c_1, c_2, c_3, \dots, c_n$ é uma cadeia ($n \geq 0$)

Condições posteriores $s = c_n, c_{n-1}, c_{n-2}, \dots, c_1$

Solução

- ▶ Caso base: $\text{reverso}(\lambda) = \lambda$
- ▶ Passo indutivo: se $c_1, c_2, c_3, \dots, c_{k-1}$ é uma cadeia de tamanho $k - 1$, para $k > 0$, o

$$\text{reverso}(c_1, c_2, c_3, \dots, c_{k-1}) = c_{k-1}, c_{k-2}, \dots, c_1$$

- ▶ Precisamos mostrar

$$\text{reverso}(c_1, c_2, c_3, \dots, c_k) = c_k, c_{k-1}, c_{k-2}, \dots, c_1$$

A verificação de algoritmos recursivos pode ser fácil a partir da indução.

Exemplo 3

Provar que um algoritmo de reverter uma cadeia está correto (2/2)

Solução

Precisamos mostrar

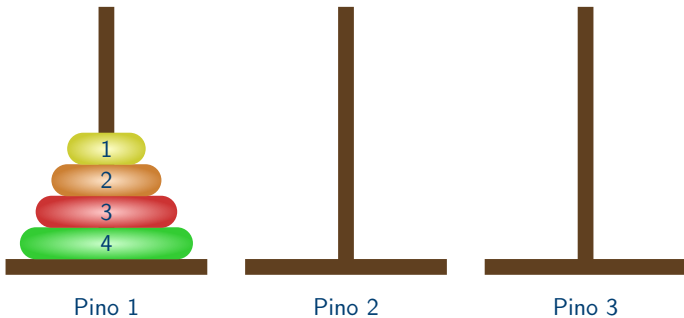
$$\text{reverso}(c_1, c_2, c_3, \dots, c_k) = c_k, c_{k-1}, c_{k-2}, \dots, c_1$$

- ▶ Se $n = 0$, então $\text{reverso}(\lambda) = \lambda$
- ▶ Se $k > 0$ e $\text{reverso}(c_1, c_2, c_3, \dots, c_{k-1}) = c_{k-1}, \dots, c_1$,
então

$$\begin{aligned}\text{reverso}(c_1, c_2, \dots, c_k) &= c_k, \text{reverso}(c_1, c_2, \dots, c_{k-1}) \\ &= c_k, c_{k-1}, c_{k-2}, \dots, c_1\end{aligned}$$

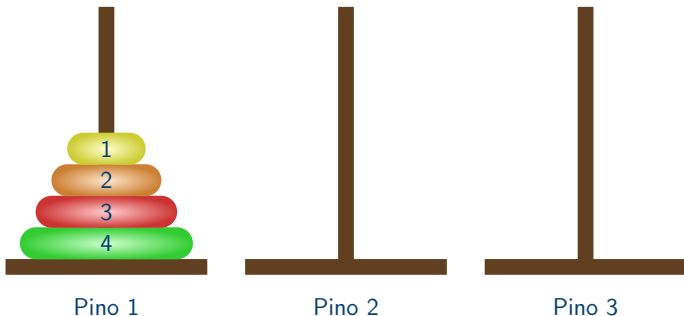
Torre de Hanoi

AS TORRES DE HANOI SÃO UM QUEBRA-CABEÇA MATEMÁTICO CLÁSSICO



Torre de Hanoi

AS TORRES DE HANOI SÃO UM QUEBRA-CABEÇA MATEMÁTICO CLÁSSICO

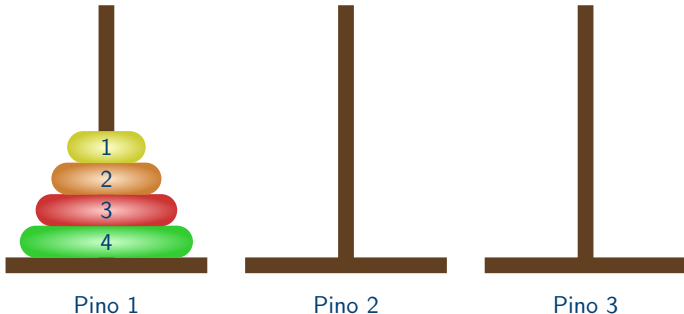


Ilustra o poder e a elegância do pensamento recursivo, e porque ele nos dá um exemplo de um algoritmo fácil de ser verificado, mas difícil de ser testado.

Torre de Hanoi

O objetivo é **mover toda a pilha de discos** para o pino mais à direita (pino 3), respeitando as seguintes regras:

- ▶ O disco do topo em qualquer um dos pinos pode ser movido para qualquer dos outros dois pinos, onde neles se torna o disco do topo.
- ▶ Apenas um disco pode ser movido por vez.
- ▶ Um disco nunca pode ser colocado em cima de um disco menor.



ESTRATÉGIA PARA RESOLVER

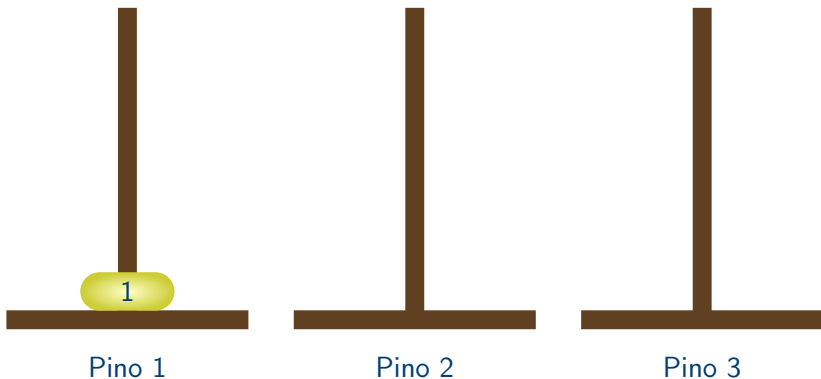
O truque para resolvermos o caso de $n = k$ discos é usar o caso $n = k - 1$ para fazer a maior parte do trabalho. Suponha que k discos estão inicialmente empilhados no pino 1.

1. Mova os discos $k - 1$ do topo do pino 1 para o pino 2, usando a solução para $n = k - 1$.
2. Mova o disco restante do pino 1 para o pino 3.
3. Use o caso $n = k - 1$ novamente para mover a pilha do pino 2 para o pino 3.

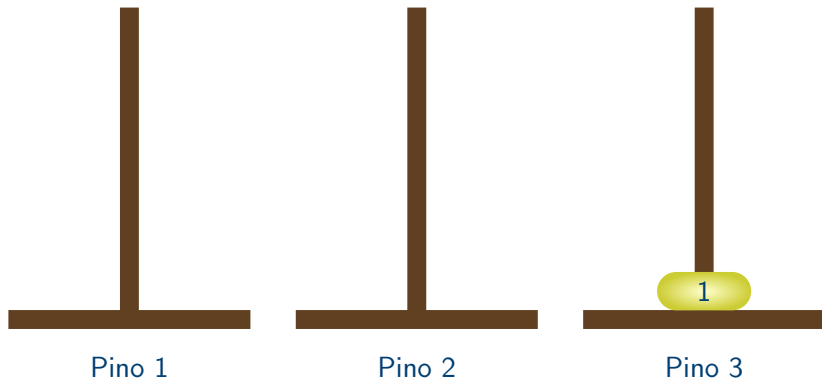
$$\begin{aligned} hanoi(2, 1, 3) &= hanoi(1, 1, 2)(1, 3)hanoi(1, 2, 3) \\ &= (1, 2)(1, 3)(2, 3) \end{aligned}$$

em que $hanoi(a, b, c)$ move a discos do pino b para o pino c .

Torre de Hanoi – 1 disco

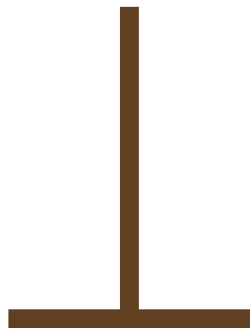


Torre de Hanoi – 1 disco



Disco movido da pino 1 para o pino 3.

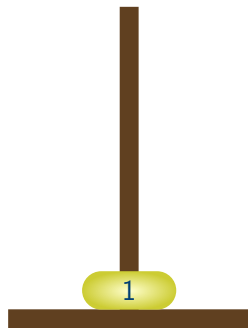
Torre de Hanoi – 1 disco



Pino 1

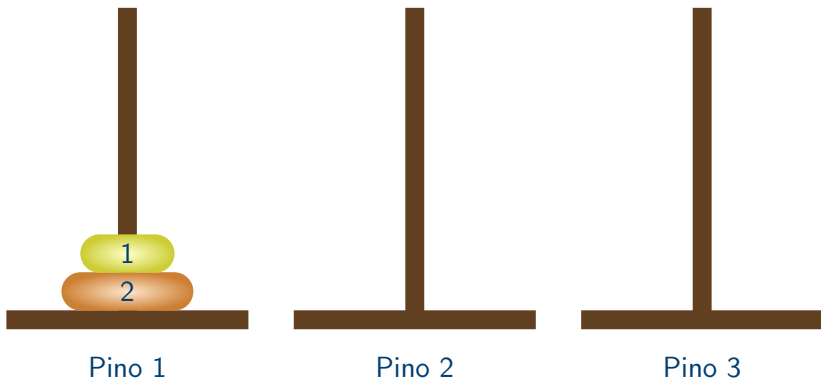


Pino 2

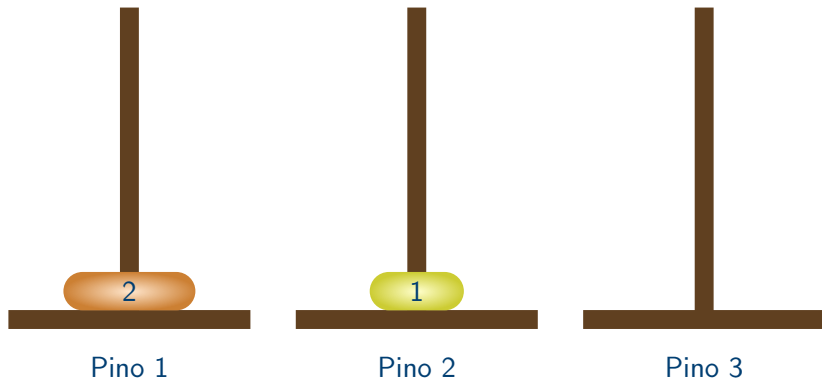


Pino 3

Torre de Hanoi – 2 discos

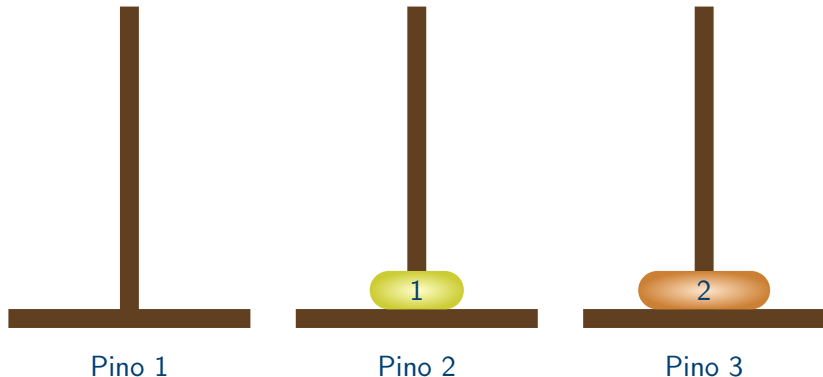


Torre de Hanoi – 2 discos



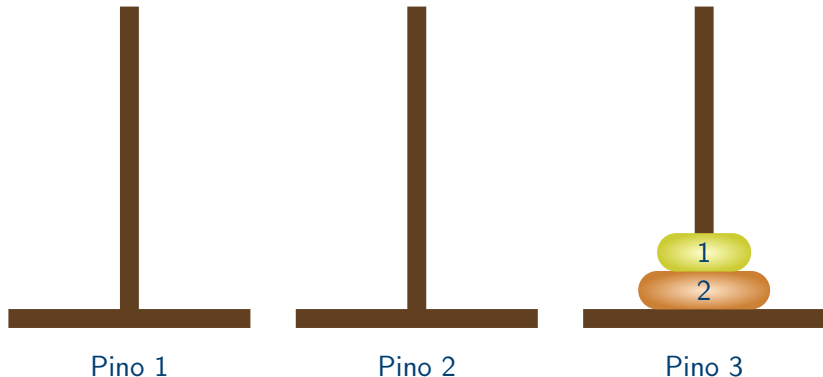
Disco movido da pino 1 para o pino 2.

Torre de Hanoi – 2 discos



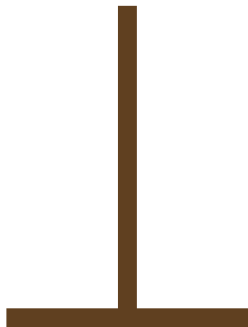
Disco movido da pino 1 para o pino 3.

Torre de Hanoi – 2 discos



Disco movido da pino 2 para o pino 3.

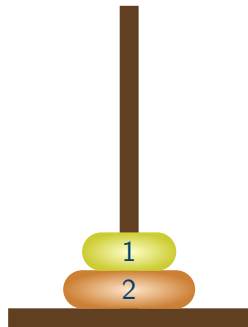
Torre de Hanoi – 2 discos



Pino 1

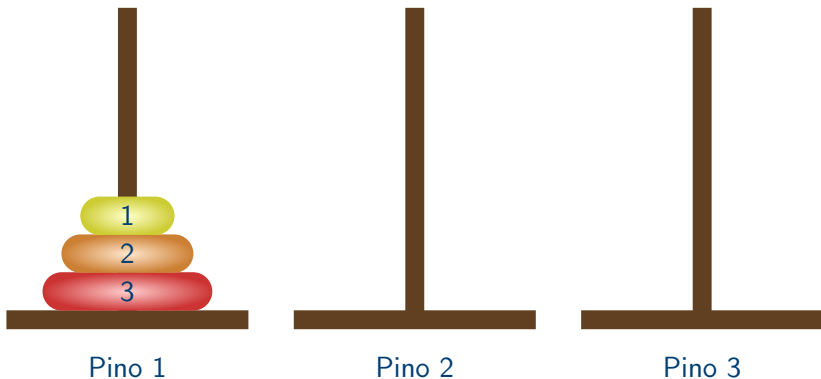


Pino 2

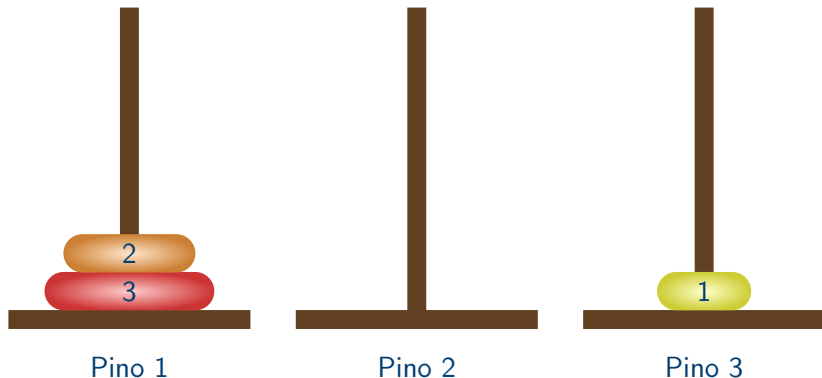


Pino 3

Torre de Hanoi – 3 discos

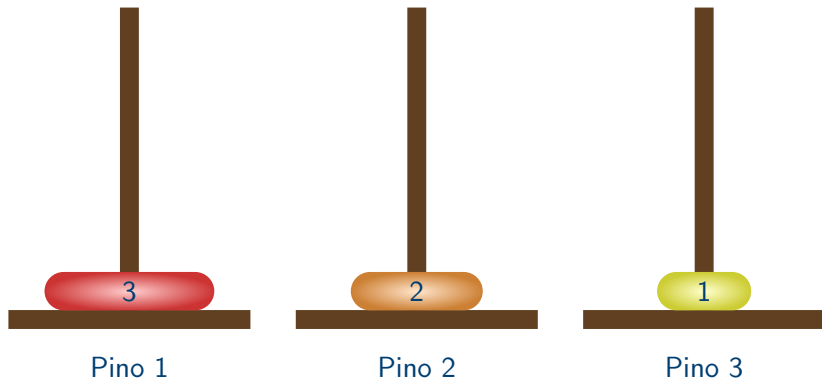


Torre de Hanoi – 3 discos



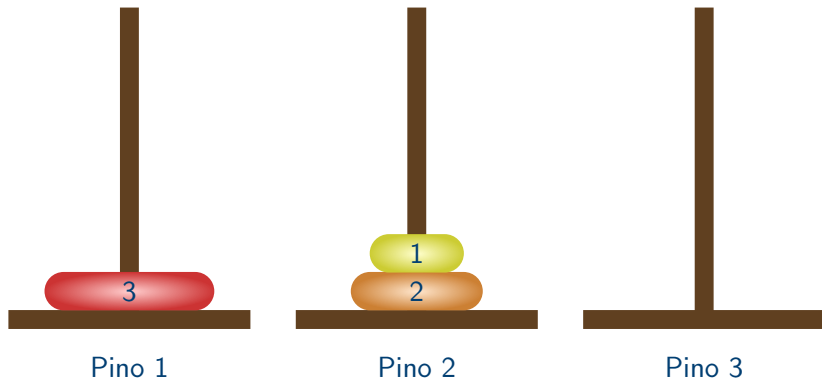
Disco movido da pino 1 para o pino 3.

Torre de Hanoi – 3 discos



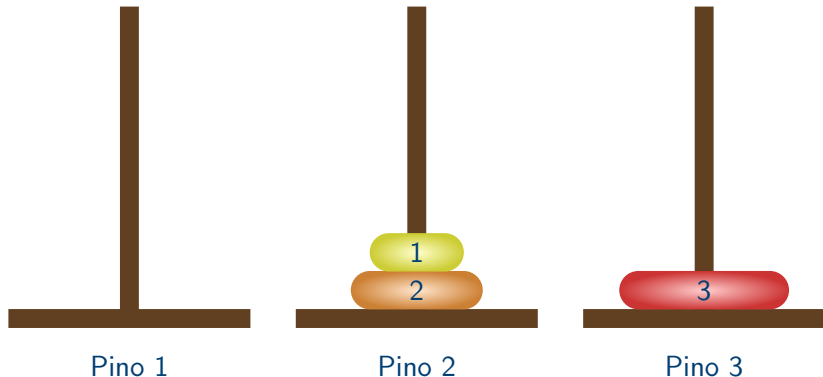
Disco movido da pino 1 para o pino 2.

Torre de Hanoi – 3 discos



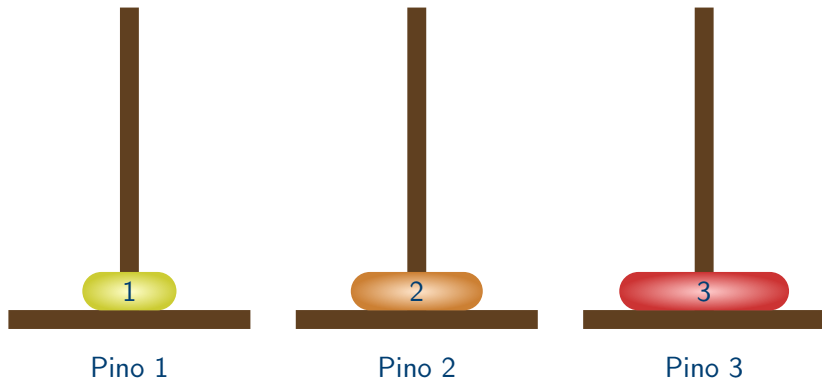
Disco movido da pino 3 para o pino 2.

Torre de Hanoi – 3 discos



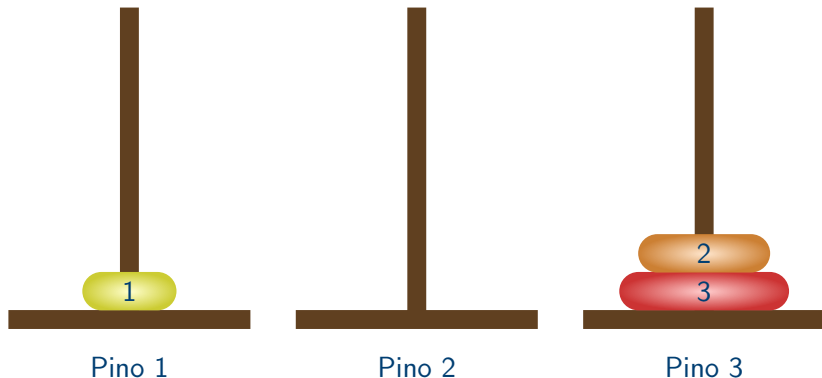
Disco movido da pino 1 para o pino 3.

Torre de Hanoi – 3 discos



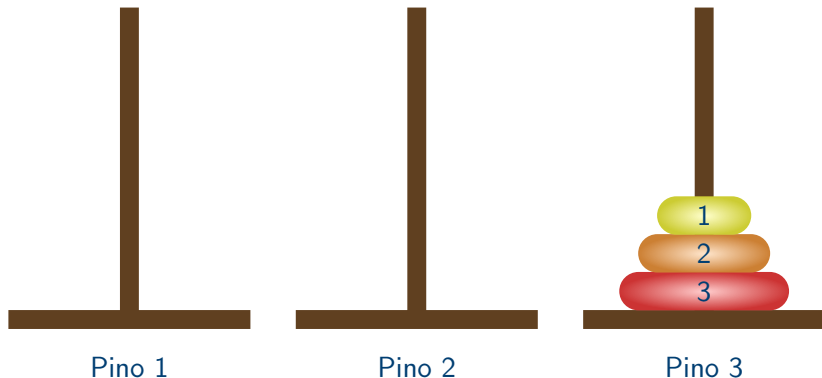
Disco movido da pino 2 para o pino 1.

Torre de Hanoi – 3 discos



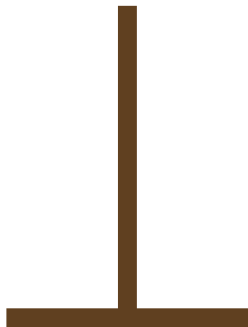
Disco movido da pino 2 para o pino 3.

Torre de Hanoi – 3 discos



Disco movido da pino 1 para o pino 3.

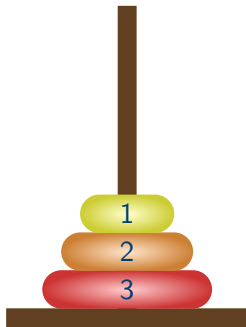
Torre de Hanoi – 3 discos



Pino 1

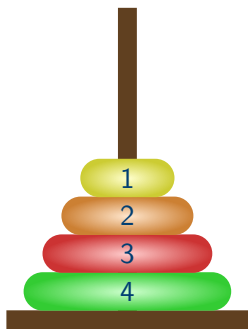


Pino 2

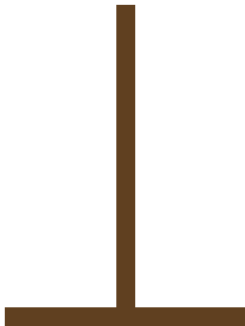


Pino 3

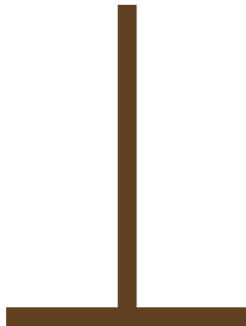
Torre de Hanoi – 4 discos



Pino 1

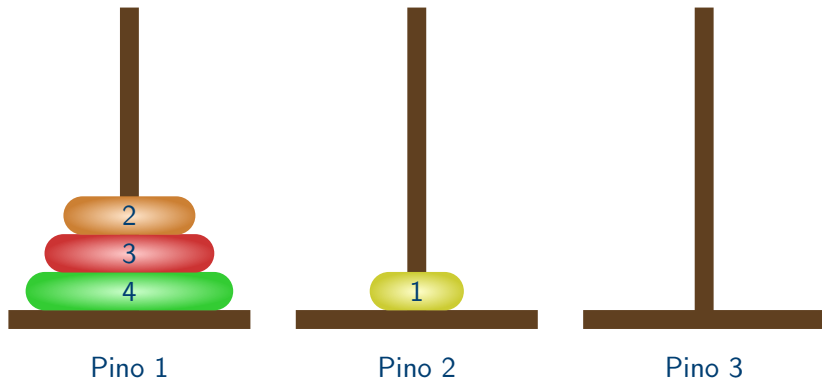


Pino 2



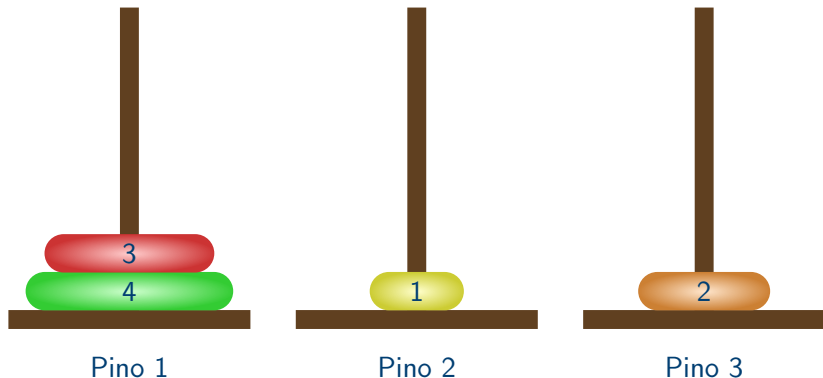
Pino 3

Torre de Hanoi – 4 discos



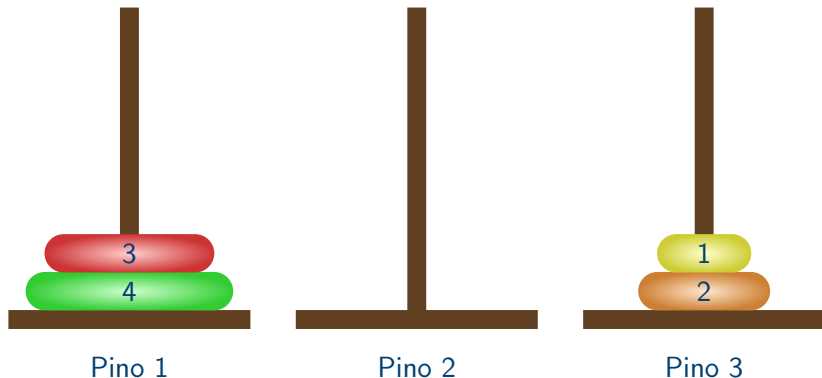
Disco movido da pino 1 para o pino 2.

Torre de Hanoi – 4 discos



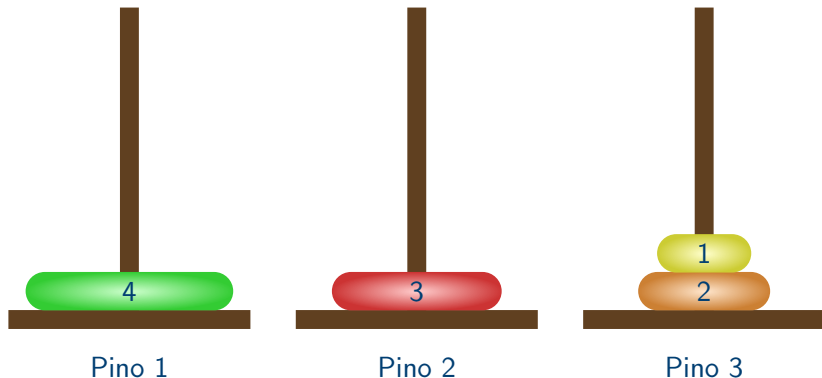
Disco movido da pino 1 para o pino 3.

Torre de Hanoi – 4 discos



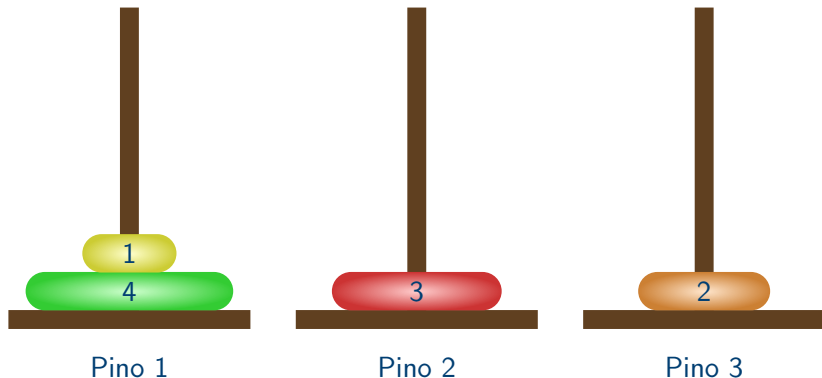
Disco movido da pino 2 para o pino 3.

Torre de Hanoi – 4 discos



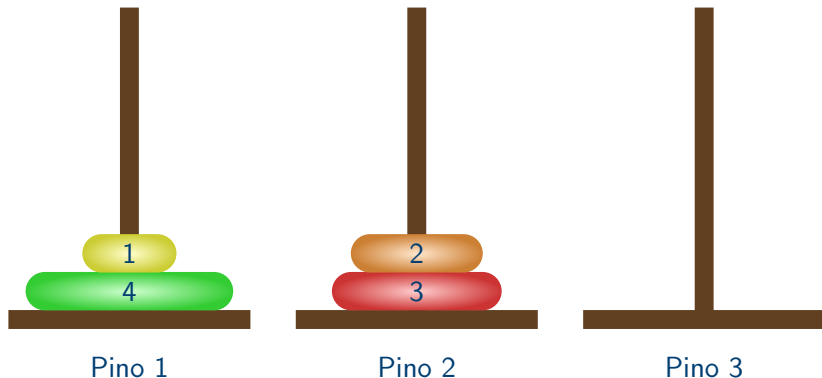
Disco movido da pino 1 para o pino 2.

Torre de Hanoi – 4 discos



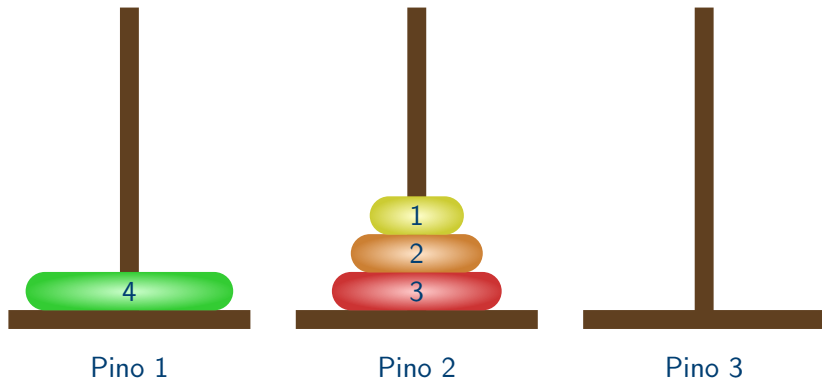
Disco movido da pino 3 para o pino 1.

Torre de Hanoi – 4 discos



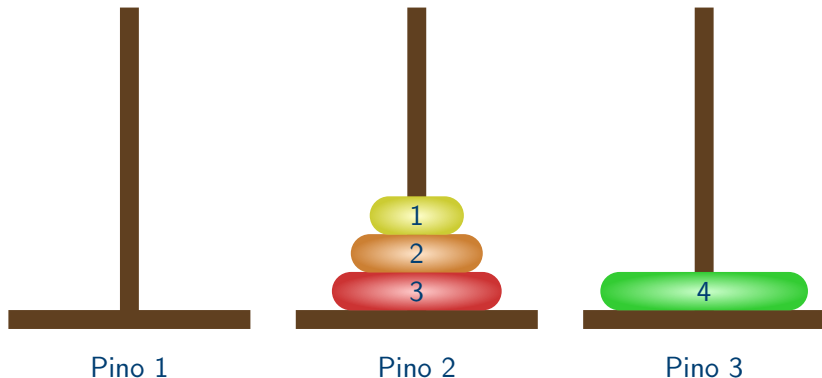
Disco movido da pino 3 para o pino 2.

Torre de Hanoi – 4 discos



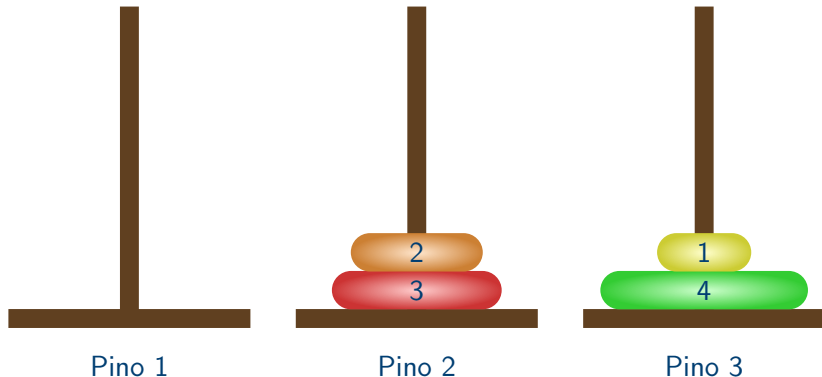
Disco movido da pino 1 para o pino 2.

Torre de Hanoi – 4 discos



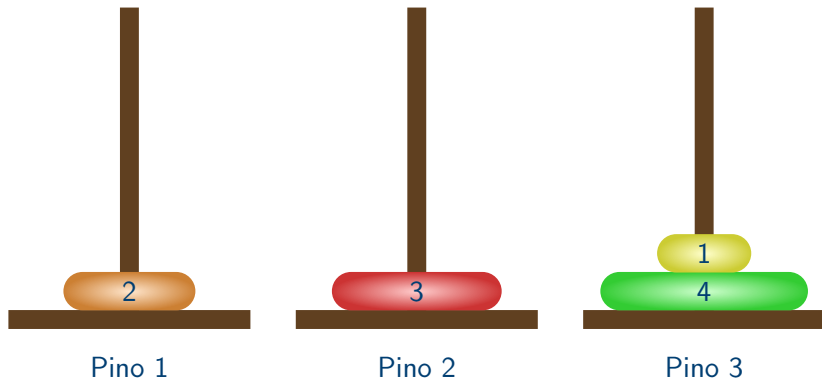
Disco movido da pino 1 para o pino 3.

Torre de Hanoi – 4 discos



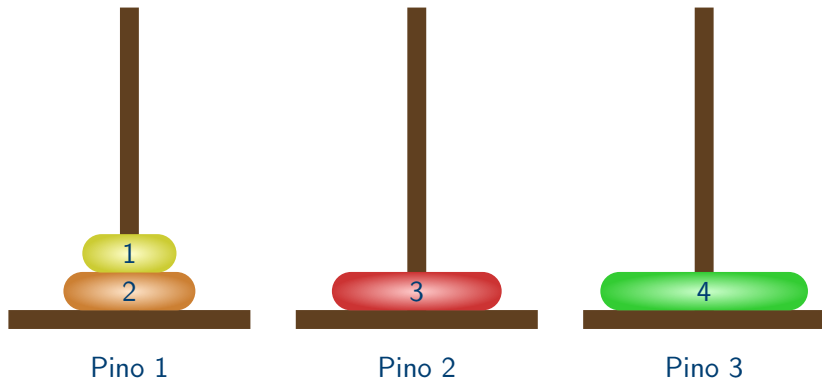
Disco movido da pino 2 para o pino 3.

Torre de Hanoi – 4 discos



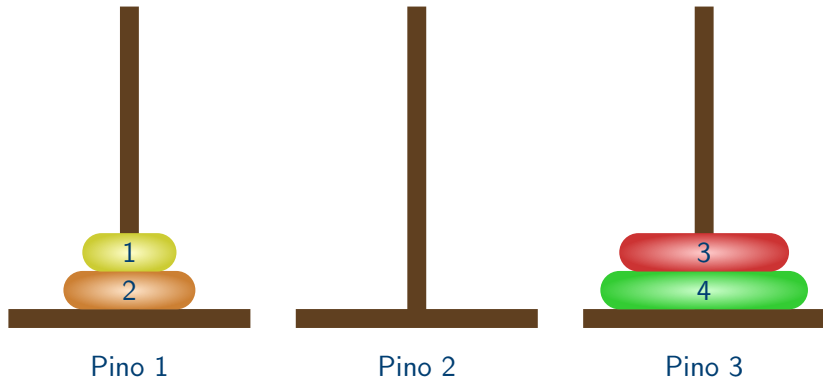
Disco movido da pino 2 para o pino 1.

Torre de Hanoi – 4 discos



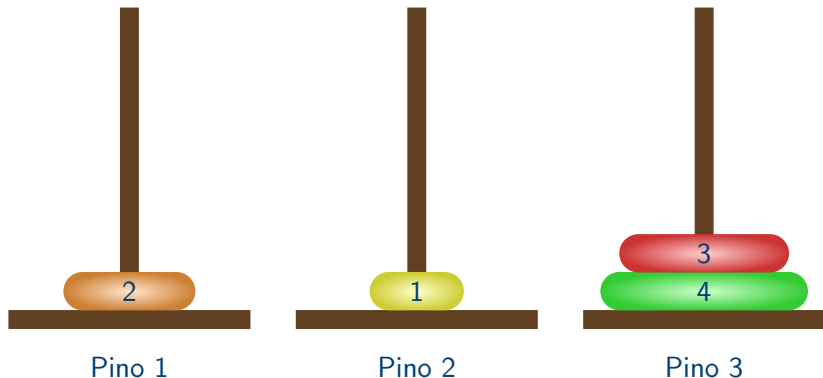
Disco movido da pino 3 para o pino 1.

Torre de Hanoi – 4 discos



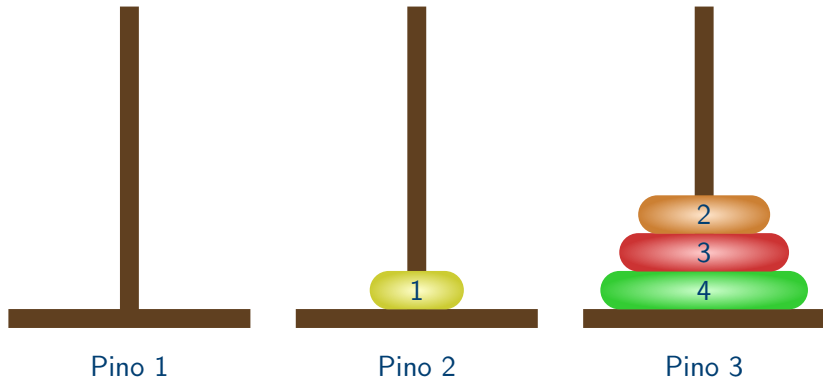
Disco movido da pino 2 para o pino 3.

Torre de Hanoi – 4 discos



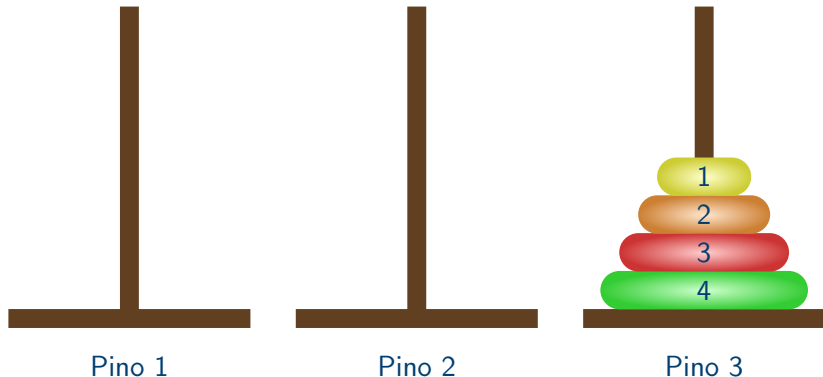
Disco movido da pino 1 para o pino 2.

Torre de Hanoi – 4 discos



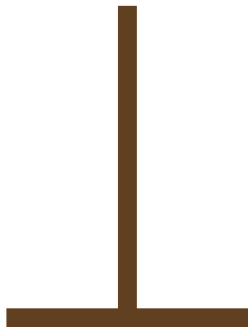
Disco movido da pino 1 para o pino 3.

Torre de Hanoi – 4 discos



Disco movido da pino 2 para o pino 3.

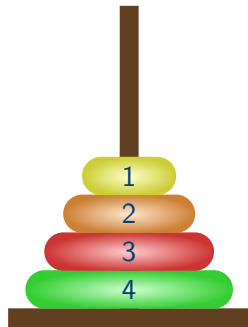
Torre de Hanoi – 4 discos



Pino 1

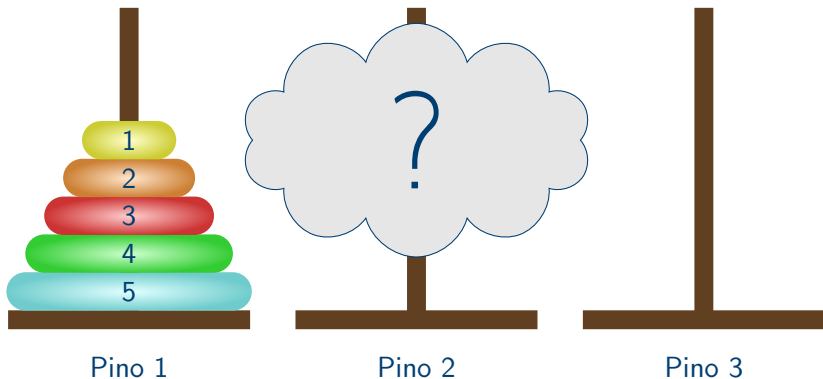


Pino 2



Pino 3

Tower of Hanoi – 5 Disc



Verificando Algoritmos Iterativos

A ideia básica por trás dos invariantes de laços é simples: se podemos isolar o que cada iteração do laço faz, então temos uma boa chance de saber o efeito acumulativo do laço.

Verificando Algoritmos Iterativos

A ideia básica por trás dos invariantes de laços é simples: se podemos isolar o que cada iteração do laço faz, então temos uma boa chance de saber o efeito acumulativo do laço.

Suponha que S representa um segmento em pseudocódigo dentro do laço. Um **invariante do laço** é um predicado P nas variáveis do programa com a propriedade de que,

- ▶ se P é verdadeiro antes de S executar, P será verdadeiro depois que S executar.
- ▶ Mais formalmente, sejam $x_1, x_2, \dots, x_n$ as variáveis do programa. $\forall i$, denote por $\underline{x_i}$ o valor da variável do programa x_i antes de S ser executado, e denote por $\overline{x_i}$ o valor de x_i após S ser executado. Então $P(x_1, x_2, \dots, x_n)$ é um invariante se

$$P(\underline{x_1}, \underline{x_2}, \dots, \underline{x_n}) \Rightarrow P(\overline{x_1}, \overline{x_2}, \dots, \overline{x_n})$$

Exemplo 4

Demonstre a correção da rotina usando o invariante de laço dado.
(1/2)

Condições prévias $m, n \in \mathbb{N}, m \geq 0, n \geq 0$

Condições posteriores $q = \lfloor m/n \rfloor, r = m \bmod n$

$q \leftarrow 0, r \leftarrow m$

// invariante: $m = qn + r, r \geq 0$

enquanto $r \geq n$ faça

$q \leftarrow q + 1$

$r \leftarrow r - n$

Exemplo 4

Demonstre a correção da rotina usando o invariante de laço dado. (2/2)

Solução Usamos indução no número de vezes que percorrermos o laço.

- ▶ Antes de o laço ser executado, $q = 0$ e $r = m$. Verificamos, então, que $m = 0 \times n + m$, o que é verdade.
- ▶ Sejam q e r os valores de q e r antes de percorrermos o laço,
- ▶ E suponha como hipótese indutiva que $m = qn + r$ com $r \geq 0$.
- ▶ Depois de mais uma iteração

$$\bar{q} = \underline{q} + 1$$

$$\bar{r} = \underline{r} - n$$

e $r \geq 0$, uma vez que $r \leq n$ era uma condição de entrada do laço.

$$\begin{aligned} m &= \underline{q}n + \underline{r} \\ &= \bar{q} - 1n + \bar{r} + n \\ &= \bar{q} + \bar{r} \end{aligned}$$

Dúvidas?

Especificação e validação formal de
– Verificação de programas –

Agradecimentos

Alguns slides foram baseados nos slides desenvolvidos por Carlos Teixeira and Daniel S. Freitas

Por favor, veja **Discrete Mathematics and Its Applications** (Kenneth H. Rosen) e **Fundamentos da matemática discreta** (David J. Hunter) para informações adicionais.